



CLI Cheatsheet - New Features

Overview

Since Nats-server v2.12 and 2.14 a number of features have been added.

Some have not (yet) been added to the standard API but are published as **orbit extensions**.

- <https://github.com/synadia-io/orbit.go>
- <https://github.com/synadia-io/orbit.java>
- <https://github.com/synadia-io/orbit.net>
- <https://github.com/synadia-io/orbit.js>
- <https://github.com/synadia-io/orbit.rs>
- <https://github.com/synadia-io/orbit.c>

New stream capabilities

See usage detailed examples in the following sections.

Per-Message TTL - Allows for auto cleaning caches and auto removal of delete markers in KVs.

```
nats s add S1 --defaults --subjects S1
--allow-msg-ttl
```

Atomic Counters in KV stores. Works across sources

```
nats s add S2 --defaults --subjects S2
--allow-counter
```

Scheduled messages - Send messages with delay or on a repeated schedule

```
nats s add S3 --defaults --subjects S3.*
--allow-schedules --allow-msg-ttl
```

Atomic batch - Commit a message batch to a stream atomically. Batch size is limited

```
nats s add S4 --defaults --subjects S4
--allow-batch
```

Fast batch - Send messages in guaranteed sequence without waiting for individual acks.

```
nats s add S5 --replicas=3 --defaults
--subjects S5 --allow-fast
```

Warning: Only activate features on streams when you intend to use them.

- Some capabilities cannot be disabled once enabled (e.g. schedules)
- Some capabilities are incompatible with Key Value Store or Source and Mirror (e.g. schedules)

Message TTL

Publish a message with different TTLs - see them disappear

```
nats pub S1 Hi --header=Nats-TTL:5s
nats pub S1 Hi --header=Nats-TTL:10s
nats pub S1 Hi --header=Nats-TTL:15s
watch -n 1 'nats s info S1'
```

Atomic Counters

Count (add) to a value atomically. Works on regular streams as well as KVs.

```
nats request S2 '' --header=Nats-Incr:+1
nats request S2 '' --header=Nats-Incr:+2
nats stream get S2 --last-for=S2'
```

Scheduled Messages

Schedule messages to be sent later.

- All scheduling happens in the same stream.
- The schedule lives in **unique** subject it is sent on. The schedule can be deleted, overwritten etc.
- The message is sent to **--schedule-dest=SUBJECT**
- The schedule may copy the last message from **--schedule-source=SUBJECT**

Schedule **once** with delay.

```
nats pub S3.1 Hi --schedule-dest=S3.s1
--schedule-at="$(date -d '+10 seconds'
-Iseconds)"
nats stream get S3 --last-for=S3.s1'
```

Scheduled message has a TTL.

```
nats pub S3.2 Hi --schedule-ttl=10s
--schedule-dest=S3.s2 --schedule-at="$(date
-d '+10 seconds' -Iseconds)"
nats stream get S3 --last-for=S3.s2'
```

Schedule **cron** style

```
nats pub S3.3 Hi --schedule-dest=S3.s3
--schedule-cron='*/5 * * * *'
nats stream get S3 --last-for=S3.s3'
```

Schedule last message from subject

```
nats pub S3.template Hello'
nats pub S3.4 Hi --schedule-dest=S3.s4
--schedule-cron='*/5 * * * *'
--schedule-source=S3.template
nats stream get S3 --last-for=S3.s4'
```

Atomic Batch Publish

Publish multiple messages in one atomic, all or nothing, operation. Limited to the same stream.

```
echo -e 'A\nB\nC\n' | nats pub --atomic
--send-on=newline S4
```

Fast Batch Publish

Allows safe (no dups, no messages lost) fast publishing in order.

```
nats bench js pub fast S5 --stream S5
```